MATH21200 (W26)
Advanced Numerical Analysis

Lecturer: Tobias Shin & Subhadip Chowdhury
Notes by: Aden Chen

Wednesday 11th March, 2026

Contents

1 Overview **4**
- 1.1 Probability . . . 4
- 1.2 Statistics . . . 4
- 1.3 Multivariate Gaussians . . . 7

2 Machine Learning and Linear Regression **9**
- 2.1 Simple Linear Regression . . . 10
- 2.2 Multivariate Linear Regression . . . 12

3 Regression Techniques **14**
- 3.1 Regularization . . . 15
- 3.2 Ridge Regression . . . 15

4 Convex Optimization **17**
- 4.1 Gradient Descent . . . 17
- 4.2 Subdifferentials . . . 18
- 4.3 Stochastic Gradient Descent . . . 19
- 4.4 Mini-batch Stochastic Gradient Descent . . . 20
- 4.5 Online Algorithms . . . 20
- 4.6 Runtime . . . 21

5 Singular Value Decomposition **22**

6 Principal Component Analysis **24**

7 QR Factorization **26**

8 Interpolation **28**
- 8.1 Newton Basis . . . 28
- 8.2 Lagrange Basis . . . 29
- 8.3 Interpolation Error Bounds . . . 29

9 Spline Interpolation **31**
- 9.1 Cubic Splines . . . 31

10 Approximation Theory **32**

11 Numerical Quadrature **34**
- 11.1 Simpson's Rule . . . 34
- 11.2 Gaussian Quadrature . . . 35
- 11.3 Quadrature on General Intervals . . . 35
- 11.4 Degree of Exactness . . . 35

12 Ordinary Differential Equations **37**
12.1 Euler's Method and Heun's Method 37
12.2 Runge-Kutta Methods 37
12.3 Multistep Methods 38

13 MISC **39**

Index **39**

Logistics (Tobias Shin):

- HW problems should not require AI.
- Coding HWs should be submitted as `.ipynb` files.
- Late HW policy: email Tobias Shin in advance ($\geq$ 3 days) with reasonable excuse.
- Midterm/final: say where/how AI was used.
- Midterm: assigned Feb 6, due Feb 9.

Logistics (Subhadip Chowdhury):

-

1 Overview

1.1 Probability

Theorem 1.1 (Law of the Unconcious Statistician (LOTUS))**.**

$$\mathbb{E}[g(X)] = \int g(x) f_X(x)\, \mathrm{d}x.$$

Some geometric intuition:

(i) $\mathbb{C}$ is bilinear (inner product).

(ii) $\mathbb{V}(X) = \mathbb{C}(X, X)$ (norm squared).

(iii) $\operatorname{sd} X \coloneqq \sqrt{\mathbb{V}(X)}$ (norm).

(iv) $\operatorname{Corr}(X, Y) \coloneqq \frac{\mathbb{C}(X,Y)}{\operatorname{sd} X \cdot \operatorname{sd} Y}$ (cosine of angle between vectors, $\langle X/\|X\|, Y/\|Y\| \rangle$).

(v) $\mathbb{E}[Y|X]$ can be thought of as the orthogonal projection of Y onto the subspace of functions of X.

(vi) Generally, $\mathbb{E}[Y|X]$ is linear in Y but not X. However, it is linear in X if X, Y are jointly normal. In this case, $\mathbb{E}[Y|X]$ is the linear regression of Y on X.

Theorem 1.2 (Law of Large Numbers)**.** *Let $X_1, \ldots$ be iid with mean $\mathbb{E}[X] = \mu$. Write the sample mean as $\overline{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$. Then $\overline{X}_n \xrightarrow{\mathrm{p}} \mu$ as $n \to \infty$.*

Theorem 1.3 (Central Limit Theorem)**.** *Let $X_1, \ldots$ be iid with mean $\mathbb{E}[X] = \mu$ and variance $\mathbb{V}(X) = \sigma^2 < \infty$. Then*

$$\frac{\sqrt{n}(\overline{X}_n - \mu)}{\sigma} \xrightarrow{\mathrm{d}} \mathcal{N}(0, 1)$$

as $n \to \infty$.

1.2 Statistics

Suppose we sample data from an experiment $X_1, X_2, \ldots$ where X_i are values of some random variable/probability distribution. Which distribution is it?

Usually, we can assume the distribution is of some nice form and determined by some parameters (**parametric statistics**).

Example 1.4. $X_i \sim \mathcal{N}(\mu, \sigma^2)$.

How do we estimate the parameters?

Example 1.5. Suppose $X_i \overset{\text{iid}}{\sim} \mathcal{N}(\mu, 1)$ and want to estimate μ.

The workflow (**maximum likelihood estimation**):

(i) Form the **likelihood** (joint pdf):

$$L(\mu, x_1, \dots, x_n) := f_{x_1,\dots,x_n}(x_1, \dots, x_n) = \prod_i f_{X_i}(x_i) = \prod_i \frac{1}{\sqrt{2\pi}} e^{-\frac{(x_i-\mu)^2}{2}}.$$

We seek the parameter μ that is most likely to have generated the data.

(ii) Optimize $L(\mu, x \dots)$ with respect to μ.

Note that it is usually easier to maximize the **log-likelihood**: $\log L$. In this case,

$$\log L = -\frac{n}{2}\log(2\pi) - \sum_i \frac{(x_i-\mu)^2}{2}.$$

Taking derivative with respect to μ and setting to zero gives

$$\hat{\mu} = \frac{1}{n}\sum_i x_i.$$

Example 1.6. Consider the exponential distribution $X \sim \text{Exponential}(\lambda)$:

$$p(x) := \lambda e^{-\lambda x}, \quad x \geq 0.$$

Suppose we see $x_1, x_2, \dots$ coming from Exponential(λ) and want to estimate λ.

The likelihood is

$$L(\lambda, x_1, \dots, x_n) = \prod_{i=1}^n \lambda e^{-\lambda x_i} = \lambda^n e^{-\lambda \sum_i x_i}.$$

The log-likelihood is

$$\log L = n \log \lambda - \lambda \sum_i x_i.$$

Taking derivative with respect to λ and setting to zero gives

$$\hat{\lambda} = \frac{n}{\sum_i x_i} = \frac{1}{\bar{x}_n}.$$

Definition 1.7. The **MLE** is the parameter value $\hat{\lambda}$ such that

$$L(\hat{\lambda}, x_1, \dots, x_n) = \max_\lambda L(\lambda, x_1, \dots, x_n).$$

Probability and Statistics:

Definition 1.8. An **estimator** is a function (thus it is a random variable) of the data used to estimate a parameter.

An estimator $\hat{\theta}$ of a parameter θ is **unbiased** if $\mathbb{E}[\hat{\theta}] = \theta$.

Probability	Statistics
$\mathbb{E}[X]$	Sample mean $\overline{X} := \frac{1}{n}\sum x_i$
$\mathbb{V}[X] := \mathbb{E}[(X-\overline{X})^2]$	Sample variance $\hat{\sigma}^2 := \frac{1}{n}\sum(x_i-\overline{X})^2$
$\mathbb{C}(X,Y) := \mathbb{E}[(X-\mathbb{E}X)(Y-\mathbb{E}Y)]$	Sample covariance $\frac{1}{n}\sum(x_i-\overline{X})(y_i-\overline{Y})$

Example 1.9.

$$\begin{aligned}
\mathbb{E}\hat{\sigma}^2 &:= \frac{1}{n}\sum \mathbb{E}(x_i^2 - 2x_i\overline{x}_i + \overline{x}^2) \\
&= \mathbb{E}\left[\frac{1}{n}\sum x_i^2 - \frac{2}{n}\overline{x}\sum x_i + \frac{\overline{x}^2}{n}n\right] \\
&= \mathbb{E}\left[\frac{1}{n}\sum x_i^2 - 2\overline{x}^2 + \overline{x}^2\right] \\
&= \mathbb{E}\left[\frac{1}{n}\sum x_i^2 - \overline{x}^2\right] \\
&= \frac{1}{n}\sum \mathbb{E}[x_i^2] - \mathbb{E}[\overline{x}^2] \\
&= \mathbb{E}[x^2] - \mathbb{E}[\overline{x}^2] \\
&= \mathbb{V}(x) + \mathbb{E}[x]^2 - \mathbb{V}(\overline{x}) - \mathbb{E}[\overline{x}]^2 \\
&= \mathbb{V}(x) - \mathbb{V}(\overline{x}) = \frac{n-1}{n}\sigma^2,
\end{aligned}$$

where we note that $\mathbb{V}(\overline{x}) = \mathbb{V}\left(\frac{1}{n}\sum x_i\right) = \frac{1}{n^2}\mathbb{V}(\sum x_i) = \sigma^2/n$. The sample variance is biased.

Remark 1.10. Some properties of an estimator of θ:

(i) **Unbiased**: $\operatorname{Bias}\hat{\theta} := \mathbb{E}[\hat{\theta}-\theta] = 0$.

(ii) **Consistent**: $\hat{\theta} \xrightarrow{\text{p}} \theta$ as $n \to \infty$. The sample mean is consistent by the LLN.

(iii) The **mean squared error** (MSE) is $\operatorname{MSE}(\hat{\theta}) := \mathbb{E}[(\hat{\theta}-\theta)^2]$.

Proposition 1.11.

$$\operatorname{MSE}(\hat{\theta}) = \mathbb{V}(\hat{\theta}) + \left(\operatorname{Bias}\hat{\theta}\right)^2.$$

From this we see a bias-variance tradeoff.

Proof. Write $\mu := \mathbb{E}[\hat{\theta}]$.

$$\begin{aligned}
\operatorname{MSE}(\hat{\theta}) &= \mathbb{E}\left[(\hat{\theta}-\mu+\mu-\theta)^2\right] \\
&= \mathbb{E}\left[(\hat{\theta}-\mu)^2 + 2(\hat{\theta}-\mu)(\mu-\theta) + (\mu-\theta)^2\right] \\
&= \mathbb{E}\left[(\hat{\theta}-\mu)^2\right] + 2(\mu-\theta)\mathbb{E}[\hat{\theta}-\mu] + (\mu-\theta)^2 \\
&= \mathbb{V}(\hat{\theta}) + \left(\operatorname{Bias}\hat{\theta}\right)^2.
\end{aligned}$$

□

1.3 Multivariate Gaussians

Definition 1.12. We say X follows a **multivariate normal distribution** with mean $\mu \in \mathbb{R}^p$ and covariance matrix $\Sigma \in \mathbb{R}^{p\times p}$ (symmetric positive semi-definite[1]), $X \sim \mathcal{N}(\mu, \Sigma)$ if its pdf is

$$p(x) = (2\pi)^{-p/2}(\det\Sigma)^{-1/2}\exp\left(-\frac{1}{2}(X-\mu)^\intercal\Sigma^{-1}(X-\mu)\right).$$

Here,

$$\mu = \mathbb{E}[X] := \begin{pmatrix}\mathbb{E}[X_1]\\ \vdots \\ \mathbb{E}[X_p]\end{pmatrix}$$

and

$$\Sigma = \mathbb{C}(X) :== \begin{pmatrix}\mathbb{V}(X_1) & \mathbb{C}(X_1,X_2) & \cdots & \mathbb{C}(X_1,X_p)\\ \mathbb{C}(X_2,X_1) & \mathbb{V}(X_2) & \cdots & \mathbb{C}(X_2,X_p)\\ \vdots & \vdots & \ddots & \vdots \\ \mathbb{C}(X_p,X_1) & \mathbb{C}(X_p,X_2) & \cdots & \mathbb{V}(X_p)\end{pmatrix}.$$

We say X, Y are **jointly Gaussian** if $[X, Y]^\intercal$ is multivariate normal.

Example 1.13. Suppose $X_1, \ldots, X_p \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$.

$$\mathbb{C}(X) = \sigma^2 I_p.$$

Theorem 1.14. *$X \in \mathbb{R}^p$ is jointly Gaussian if and only if* every *linear combination of the components is univariate Gaussian, i.e., for all $a \in \mathbb{R}^p$, $a_1X_1 + \cdots + a_pX_p$ is Gaussian.*

NB: all X_i normal does not imply jointly Gaussian. Though independent normal random variables are jointly normal.

Theorem 1.15. *Let $X \in \mathbb{R}^p$ and $Y \in \mathbb{R}^q$ be jointly Gaussian. Then $\mathbb{C}(X, Y) = 0$ if and only if X and Y are independent ($X \perp\!\!\!\perp Y$).*

Proof. For the forward direction, we have

$$\begin{pmatrix}X\\Y\end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix}\mu_X\\ \mu_Y\end{pmatrix}, \begin{pmatrix}\Sigma_X & 0\\ 0 & \Sigma_Y\end{pmatrix}\right).$$

Thus (using the fact that $\det\begin{pmatrix}A & \\ & B\end{pmatrix} = \det A \cdot \det B$),

$$\begin{aligned}
p(x,y) &= (2\pi)^{-(p+q)/2}(\det\Sigma_X)^{-1/2}(\det\Sigma_Y)^{-1/2}\exp\left(-\frac{1}{2}\begin{pmatrix}x^\intercal - \mu_X^\intercal & y^\intercal - \mu_Y^\intercal\end{pmatrix}\begin{pmatrix}\Sigma_X^{-1} & 0\\ 0 & \Sigma_Y^{-1}\end{pmatrix}\begin{pmatrix}x-\mu_X\\ y-\mu_Y\end{pmatrix}\right)\\
&= (2\pi)^{-(p+q)/2}(\det\Sigma_X)^{-1/2}(\det\Sigma_Y)^{-1/2}\exp\left(-\frac{1}{2}(x-\mu_X)^\intercal\Sigma_X^{-1}(x-\mu_X) - \frac{1}{2}(y-\mu_Y)^\intercal\Sigma_Y^{-1}(y-\mu_Y)\right)\\
&= p_X(x)\cdot p_Y(y).
\end{aligned}$$

[1] I.e., all eigenvalues (which it has, being symmetric) are nonnegative.

The reverse direction is similar. □

2 Machine Learning and Linear Regression

What is machine learning?

- Finding patterns in data, used to make predictions.
- Using models and statistics to understand these patterns.
- Optimization algorithms to "learn" the patterns.

Two types of problems in basic machine learning:

- **Classification**: given a set of data with labels $\{x_i, y_i\}$ specifying a **class** the data point is in.
  - Here, x_i are called the predictors/features/independent variables, and y_i are called labels/response/dependent variables.

  We try to predict the class of a new data point.
- **Regression**: Given a set of data points with numeric labels, we try to predict future numeric values.

ML abstractions:

(i) Application/Data: is the data labeled (supervised vs. unsupervised)?
  - If yes, are the labels categorical (classification) or numeric (regression)?
  - If no, is the goal to find similar structures (clustering) or reduce complexity (dimensionality reduction)?

(ii) Model (say Classification):
  - Decision functions (linear, polynomial, logistic, neural nets).
  - Tree-based models
  - Linear models

(iii) Optimization Problem:
  - Loss function to optimize.
  - Constraints
  - Variables, features

(iv) Algorithms: Gradient descent, SVD, simplex method, PCA, etc.

Example 2.1. Given a set of labeled data, (x_i, y_i), say x_i is the size of the house and y_i is the price of house. How to predict house price based on size?

2.1 Simple Linear Regression

We are given labeled data (x_i, y_i).

Assumption 2.2 (Linear Regression). Assume

$$y_i = x_i\beta_1 + \beta_0 + \sigma z_i,$$

where β_1 is the slope, β_0 is the intercept, σ is the noise scale, and z_i (random) is noise and assumed to be iid $\mathcal{N}(0, 1)$ with the same variance (homoscedasticity). We view x_i (as well as β_0, β_1, σ) as observed/fixed and z_i (and thus y_i) as random. Note that we have $y_i \sim \mathcal{N}(x_i\beta_1 + \beta_0, \sigma^2)$.

How to estimate β_1 and β_0? Use MLE.

Write $\beta \coloneqq (\beta_1, \beta_0)$. We have

$$p(\beta; x_i, y_i) = \prod_n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y_i-\beta_0-\beta_1 x)^2}{\sigma^2}}$$
$$\log p(\beta; x_i, y_i) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Maximizing the log-likelihood is equivalent to minimizing

$$S(\beta) \coloneqq \sum_n (\beta_1 x_i + \beta_0 - y_i)^2.$$

This is the **least squares error**.

The first order conditions are

$$\frac{\partial S}{\partial \beta_0} = 2\sum_n (y_i - \beta_0 - \beta_1 x_i) = 0,$$
$$\frac{\partial S}{\partial \beta_1} = 2\sum_n x_i(y_i - \beta_0 - \beta_1 x_i) = 0$$

which give

$$\beta_0 + \beta_1\overline{x} = \overline{y}$$
$$\beta_0\overline{x} + \beta_1 \sum \frac{1}{n}x_i^2 = \frac{1}{n}\sum x_i y_i.$$

Subtracting $\overline{x}\times$ (first equation) from the second gives

$$\left[\frac{1}{n}\sum x_i^2 - \left(\frac{1}{n}\sum x_i\right)^2\right]\beta_1 = \frac{1}{n}\sum x_i y_i - \overline{x}\cdot\overline{y}$$

and thus

$$\begin{aligned}\hat{\beta}_1 &= \frac{\frac{1}{n}\sum x_i y_i - \left(\frac{1}{n}\sum x_i\right)\left(\frac{1}{n}\sum y_i\right)}{\frac{1}{n}\sum x_i^2 - \left(\frac{1}{n}\sum x_i\right)^2} \\ &= \frac{\mathbb{E}(XY) - \mathbb{E}(X)\mathbb{E}(Y)}{\mathbb{E}[X^2] - (\mathbb{E}X)^2} = \frac{\mathbb{C}(X,Y)}{\mathbb{V}(X)} \\ &= \frac{\frac{1}{n}\sum(x_i - \overline{x})(y_i - \overline{y})}{\frac{1}{n}\sum(x_i - \overline{x})^2} = \frac{\frac{1}{n}\sum(x_i - \overline{x})y_i}{\frac{1}{n}\sum(x_i - \overline{x})^2},\end{aligned}$$

where the expectations/covariance/variance are the sample versions. The equalities in the last line are left as an exercise. This also gives $\hat{\beta}_0$ as

$$\hat{\beta}_0 = \overline{y} - \hat{\beta}_1\overline{x}.$$

We next treat y_i as random and x_i as fixed and derive the distribution of $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1)$. We start with their expectations:

$$\mathbb{E}[\hat{\beta}_1] = \frac{\frac{1}{n}\sum(x_i - \overline{x})\mathbb{E}[y_i - \overline{y}]}{\frac{1}{n}\sum(x_i - \overline{x})^2} = \frac{\frac{1}{n}\sum(x_i - \overline{x})(x_i\beta_1 + \beta_0 - \overline{x}\beta_1 - \beta_0)}{\frac{1}{n}\sum(x_i - \overline{x})^2} = \beta_1.$$

This also gives

$$\mathbb{E}[\hat{\beta}_0] = \mathbb{E}[\overline{y}] - \overline{x}\mathbb{E}[\hat{\beta}_1] = \overline{x}\beta_1 + \beta_0 - \overline{x}\beta_1 = \beta_0.$$

Thus $\hat{\beta} := (\hat{\beta}_1, \hat{\beta}_0)$ is unbiased.

Next, we compute their variances: Using $\hat{\beta}_1 = \frac{\sum(x_i - \overline{x})y_i}{\sum(x_i - \overline{x})^2}$, we have

$$\mathbb{V}[\hat{\beta}_1] = \frac{\sum(x_i - \overline{x})^2\,\mathbb{V}(y_i)}{\left(\sum(x_i - \overline{x})^2\right)^2} = \frac{\sum(x_i - \overline{x})^2}{\left(\sum(x_i - \overline{x})^2\right)^2}\sigma^2 = \frac{\sigma^2}{\sum(x_i - \overline{x})^2} = \frac{\sigma^2}{n}\cdot\frac{1}{\mathbb{V}(X)}.$$

The variance increases as the noise σ^2 increases, and decreases as the spread of the x_i (i.e., $\mathbb{V}(X)$) increases or the number of data points n increases. Using this, we compute the variance of $\hat{\beta}_0$:

$$\begin{aligned}\mathbb{V}(\hat{\beta}_0) &= \mathbb{V}(\overline{y} - \hat{\beta}_1\overline{x}) = \mathbb{V}(\overline{y}) + \overline{x}^2\,\mathbb{V}(\hat{\beta}_1) - 2\overline{x}\mathbb{C}(\overline{y}, \hat{\beta}_1) \\ &= \frac{\sigma^2}{n} + \overline{x}^2\cdot\frac{\sigma^2}{n}\cdot\frac{1}{\mathbb{V}(x)} + 0 = \frac{\sigma^2}{n}\left(1 + \frac{\overline{x}^2}{\mathbb{V}(X)}\right).\end{aligned}$$

Here we used the fact that $\mathbb{C}(\overline{y}, \hat{\beta}_1) = 0$.

Proof.

$$\mathbb{C}\left(\overline{y}, \frac{\sum(x_i - \overline{x})y_i}{\sum(x_i - \overline{x})^2}\right) = \frac{\sum(x_i - \overline{x})\mathbb{C}(\overline{y}, y_i)}{\sum(x_i - \overline{x})^2} = \frac{\sum(x_i - \overline{x})}{\sum(x_i - \overline{x})^2}\cdot\frac{\sigma^2}{n} = 0,$$

where we used that for each i,

$$\mathbb{C}\left(\frac{1}{n}\sum y_j, y_i\right) = \frac{\sigma^2}{n}.$$

□

We conclude with computing $\mathbb{C}(\hat{\beta}_1, \hat{\beta}_0)$:

$$\begin{aligned}\mathbb{C}(\hat{\beta}_1, \hat{\beta}_0) &= \mathbb{C}(\hat{\beta}_1, \overline{y} - \hat{\beta}_1 \overline{x}) = \mathbb{C}(\hat{\beta}_1, \overline{y}) - \overline{x}\,\mathbb{V}(\hat{\beta}_1) \\ &= 0 - \frac{\sigma^2}{n} \cdot \frac{\overline{x}}{\mathbb{V}(X)}.\end{aligned}$$

2.2 Multivariate Linear Regression

Model: $y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_{p-1} x_{ip-1} + \sigma z_i$.

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p-1} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np-1} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{p-1} \end{pmatrix} + \sigma \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{pmatrix}.$$

Note that the four matrices are of size $n \times 1$, $n \times p$, $p \times 1$, and $n \times 1$ respectively. We can write this concisely as $y = X\beta + \delta Z$, where $Z \sim \mathcal{N}(0, \mathrm{Id}_{n\times n})$, or $y \sim \mathcal{N}(X\beta, \sigma^2\,\mathrm{Id})$.

We calculate the maximum likelihood estimator $\hat{\beta}$:

$$L(\beta, X, y) = p(y) \propto \exp\left(-\frac{1}{2\sigma^2}(y - X\beta)^\intercal (y - X\beta)\right) = \exp\left(-\frac{1}{2\sigma^2}\|y - X\beta\|^2\right).$$

Thus maximizing the likelihood is equivalent to minimizing the least squares error. Now,

$$\begin{aligned}\log L(\beta, X, y) &= -\frac{1}{2\sigma^2}\|y - X\beta\|^2 \\ &= -\frac{1}{2\sigma^2}(y^\intercal y - 2y^\intercal X\beta + \beta^\intercal X^\intercal X\beta).\end{aligned}$$

Thus,

$$\nabla_\beta \log L = -\frac{1}{2\sigma^2}(-2X^\intercal y + 2X^\intercal X\beta),$$

which gives $X^\intercal X\hat{\beta} = X^\intercal y$ and then

$$\hat{\beta} = (X^\intercal X)^{-1} X^\intercal y.$$

Note that we imposed the following

Assumption 2.3.

- Linear model: $Y = X\beta + \sigma Z$.
- Normal and homoscedasticity errors: $Z \sim \mathcal{N}(0, \mathrm{Id})$.
- No multicollinearity: columns (features) of X are linearly independent (so $X^\intercal X$ is invertible).

Bias of $\hat{\beta}$:

$$\mathbb{E}\hat{\beta} = (X^\intercal X)^{-1} X^\intercal \mathbb{E}[y] = (X^\intercal X)^{-1} X^\intercal X\beta = \beta.$$

Variance of $\hat{\beta}$:

$$\mathbb{C}(\hat{\beta}) = (X^\intercal X)^{-1} X^\intercal \mathbb{C}(y) X (X^\intercal X)^{-1} = \sigma^2 (X^\intercal X)^{-1}.$$

2.2.1 Projection

Definition 2.4. A matrix $P \in \mathbb{R}^{n\times m}$ is a **projection matrix** if $P^2 = P$. It is an **orthogonal projection** if, in addition to being a projection, it is symmetric.

Example 2.5.

- $P = X(X^\top X)^{-1}X^\top$ is an orthogonal projection on the column space of X.
- Let $u \in \mathbb{R}^n$ be any unit vector, i.e., $\|u\| = u^\top u = 1$. Then $P := uu^\top$ is a projection to u.
- Averaging is a projection. Let $X = \begin{pmatrix}1 \dots 1\end{pmatrix}^\top$. Then $p = (X^\top X)^{-1}Y = n^{-1}\sum Y_i$.

Proposition 2.6. *Let P be an orthogonal projection. Then*

(i) $\mathrm{Id} - P$ *is an orthogonal projection.*

(ii) The eigenvalues of P are either 0 *or* 1.

Proof. Since P is symmetric we can write $P = U\Lambda U^\top$. Since P is a projection,$P^2 = U\Lambda^2 U^\top = U\Lambda U^\top = P$. This gives $\Lambda^2 = \Lambda$, so the eigenvalues are 0 or 1. □

(iii) If P_1, P_2 are projections such that $P_1P_2 = 0$, then $P_1 + P_2$ is a projection.

(iv) If $P \in \mathbb{R}^{n\times n}$, then $\mathrm{rank}(P) = \mathrm{tr}(P) = \sum \lambda_i = r$, $\mathrm{rank}(\mathrm{Id} - P) = n - r$.

Proof. $\mathrm{tr}(P) = \mathrm{tr}(U\Lambda U^\top) = \mathrm{tr}(\Lambda U^\top U) = \mathrm{tr}(\Lambda)$. □

2.2.2 Model Accuracy

We can decompose the total variance in data into variance explained by the model and residual variance:

$$\underbrace{\|y - \overline{y}\|^2}_{TSS} = \underbrace{\|y - \hat{y}\|^2}_{RSS} + \underbrace{\|\hat{y} - \overline{y}\|^2}_{ESS/MSS}.$$

We define

$$R^2 := 1 - \frac{RSS}{TSS} = \frac{ESS}{TSS}.$$

In the univariate case,

$$\hat{\beta} = \frac{\mathbb{C}(X,Y)}{\mathbb{V}(X)} = \frac{\mathrm{Corr}(X,Y)\,\mathrm{sd}\,Y}{\mathrm{sd}\,X}, \quad \mathrm{Corr}(X,Y) = \frac{\mathbb{C}(X,Y)}{\mathrm{sd}\,X\,\mathrm{sd}\,Y} = \cos\theta = \frac{\|\hat{y} - \overline{y}\|}{y - \overline{y}}.$$

Thus,

$$\mathrm{Corr}^2(X,Y) = \frac{\|\hat{y} - \overline{y}\|^2}{\|y - \overline{y}\|^2} = R^2.$$

3 Regression Techniques

OLS linear regression: $y = X\beta + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \mathrm{Id})$.

$$\begin{aligned}
\hat{\beta} &= (X^\top X)^{-1} X^\top y \\
\hat{y} &= X\hat{\beta}, \\
\mathbb{E}[\hat{\beta}] &= \beta, \\
\mathbb{C}(\hat{\beta}) &= \sigma^2 (X^\top X)^{-1}.
\end{aligned}$$

Recall also the bias-variance decomposition:

$$\mathrm{MSE}(\hat{\theta}) = \mathbb{V}(\hat{\theta}) + \left(\mathrm{Bias}\,\hat{\theta}\right)^2.$$

Applying to $\hat{y}_i$ as a predictor for $\theta := x_i\beta_1 + \beta_0$ gives

$$\mathbb{E}\left[(\hat{y}_i - (x_i\beta_1 + \beta_0))^2\right] = \mathbb{E}[(\hat{y}_i - \mu)^2] + (\mu - \theta)^2,$$

where $\mu = \mathbb{E}[\hat{y}_i] = x_i\beta_1 + \beta_0 = \theta$. This is a low bias, high variance model. What to do?

In general in ML, a model can suffer from **underfitting**: model too simple. Signs of underfitting include:

- Dominant error comes from bias.
- High error on training data.
- High error on testing data.

A model can also suffer from **overfitting**: model too complex. Signs include:

- Low error on training data.
- High error on testing data.
- Dominant error comes from variance.

Remark 3.1 (Three Kinds of Data).

(i) Training data.

(ii) Validation data.

(iii) Testing data.

When making a model, you train the model on (i). Take say 80% of (i) + (ii) to be (i) and 20% to be (ii).

k-fold cross validation: Split data into k parts. For each part, train on the other $k - 1$ parts and validate on that part. ☕

3.1 Regularization

Recall that

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|^2.$$

3.2 Ridge Regression

We optimize

$$\hat{\beta}_{\text{ridge}} = \arg\min_{\beta} \left(\|y - X\beta\|^2 + \lambda \|\beta\|^2 \right),$$

where $\lambda > 0$ is a regularization parameter.

- This reduces to OLS when $\lambda = 0$.
- The second term penalizes large values of β. Thus one should normalize features before applying ridge regression (same units).

The objective is

$$\begin{aligned} \|y - X\beta\|^2 + \lambda \|\beta\|^2 &= \|y\|^2 + \|X\beta\|^2 - 2y^\top X\beta + \lambda \|\beta\|^2 \\ &= y^\top y - 2y^\top X\beta + \beta^\top (X^\top X + \lambda \operatorname{Id})\beta. \end{aligned}$$

Thus,

$$\nabla_\beta = -2X^\top y + 2(X^\top X + \lambda \operatorname{Id})\beta,$$

giving

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda \operatorname{Id})^{-1} X^\top y.$$

Exercise 3.2. Why can we invert $X^\top X + \lambda \operatorname{Id}$?

3.2.1 Bayesian POV

We're discussed $y = X\beta + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Before, we derived $\hat{\beta}$ for linear regression using MLE.

Remark 3.3 (Frequentists vs Bayesians).

- Frequentists: probability of statistical event measured in its frequency according to some *fixed* background *distribution* with *parameters*.
- Bayesians: assume *prior* distribution observed and *update* distribution with new data using Bayes theorem.

Now assume β is also a random variable with prior $\beta \sim \mathcal{N}(0, \tau^2 \operatorname{Id})$. Using Bayes theorem, we have

$$p(\beta|y, x) = \frac{p(y|\beta, x)p(\beta)}{p(y|x)} \propto p(y|\beta, x)p(\beta) = e^{-\frac{\|y - X\beta\|^2}{2\sigma^2}} e^{-\frac{\|\beta\|^2}{2\tau^2}}.$$

Now maximize posterior with respect to β (this is called the **maximum a posteriori** (MAP) estimate):

$$\frac{\mathrm{d}}{\mathrm{d}\beta} \log p(\beta|y,x) = \frac{\mathrm{d}}{\mathrm{d}\beta}\left(-\frac{\|y - X\beta\|^2}{2\sigma^2} - \frac{\|\beta\|^2}{2\tau^2}\right).$$

This is equivalent to minimizing

$$\|y - X\beta\|^2 + \frac{\sigma^2}{\tau^2}\|\beta\|^2,$$

which is exactly ridge regression with $\lambda = \sigma^2/\tau^2$.

Note that $\tau^2 = 0$ is equivalent to β can be anything (uninformed prior), which gives back OLS.

Exercise 3.4. X random variable with variance 0 if and only if X is almost surely a constant.

3.2.2 LASSO Regression

LASSO = Least Absolute Shrinkage and Selection Operator.

We optimize

$$\hat{\beta}_{\text{lasso}} = \arg\min_{\beta}\left(\|y - X\beta\|^2 + \lambda\|\beta\|_1\right),$$

where $\|\beta\|_1 = \sum |\beta_i|$.

Remark 3.5 (Unit balls in L^p spaces). The unit ball in L^p is convex for $p \geq 1$. For $p < 1$ it is concave. ☕

We can interpret LASSO regression as optimizing $\|y - X\beta\|^2$ with the constraint $\|\beta\|_1 \leq C$. Note that at times the solution will have some $\beta_i = 0$, i.e., feature selection.

There is no closed form solution for LASSO regression. One has to solve it numerically.

4 Convex Optimization

We say f is **convex** if for each x, y and $t \in [0, 1]$,

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y).$$

Theorem 4.1. *If f is convex, then any local minimum is a global minimum.*

4.1 Gradient Descent

We set

$$x^{t+1} := x^t - \eta f'(x^t)$$

until $|x^{t+1} - x^t| < \varepsilon$ for some $\varepsilon > 0$ The parameter $\eta > 0$ is the step size.

Usually, we pick η to be inversely proportional to the curvature of the graph of f. In practice, we start with a large η. If the algorithm does not converge, we divide η by 2 and try again.

Starting with a small η will converge but takes longer.

Example 4.2 (Linear Regression). $f(\beta) := \|y - X\beta\|^2$. $\nabla_\beta f = -2X^\top(y - X\beta)$.

(i) Initialize β_0, η.

(ii) For $k = 0, 1, \ldots$ until convergence:

- $r := y - X\beta_k$.
- $g = -2X^\top r$.
- $\beta_{k+1} := \beta_k - \eta g$.

Note that each if the three steps takes $O(np)$ time.

Compare the run time of this with that of explicitly computing $\hat{\beta} = (X^\top X)^{-1}X^\top y$, which is at least cubic in p due to matrix inversion. ☙

Example 4.3.

$$f(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2 = \|y\|^2 - \beta^\top(X^\top X + \lambda \operatorname{Id})\beta - 2\beta^\top X^\top y.$$

Thus, $\nabla_\beta f = 2(X^\top X + \lambda \operatorname{Id}) - 2X^\top y$.

The algorithm proceeds as follows:

- Set $\beta^0 = 0$.
- Set
$$\beta^{t+1} := \beta^\top - 2\eta\left[(X^\top X + \lambda \operatorname{Id})\beta^\top - X^\top y\right]$$
until convergence.

Note that we are not computing inverses here, which is computationally expensive. Note also that the larger λ is, the faster the convergence. ☙

Example 4.4.

$$f(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|_1.$$

But $\|\cdot\|_1$ is not differentiable everywhere. What to do? ☙

4.2 Subdifferentials

Definition 4.5. A **subderivative** of a convex function $f : I \to \mathbb{R}$ at x_0 is a real number c such that $f(x) - f(c) \geq c(x - x_0)$.

Proposition 4.6. *The set of subderivatives at x_0 for a convex f is a closed interval $[a, b]$, where*

$$a := \lim_{x \to x_0^-} \frac{f(x) - f(x_0)}{x - x_0}, \quad b := \lim_{x \to x_0^+} \frac{f(x) - f(x_0)}{x - x_0}.$$

*The interval $[a, b]$ is the **subdifferential** of f at x_0, denoted $\partial f(x_0)$.*

Proof. Left as an exercise. □

Exercise 4.7. f is differentiable at x_0 if and only if $\partial f(x_0) = \{f'(x_0)\}$.

Proposition 4.8. *x_0 is a minimum for f convex if and only if $0 \in \partial f(x_0)$.*

Example 4.9. $f(\beta) = (\beta - z)^2 + \lambda|\beta|$.

$$\partial f(\beta) = 2(\beta - z) + \lambda \partial|\beta|.$$

- If $\beta > 0$, we have $\partial|\beta| = \{1\}$ and thus set

  $$0 = \partial f(\beta) = 2(\beta - z) + \lambda,$$

  which gives $\beta = z - \lambda/2$ (this is valid if $z > \lambda/2$).

- If $\beta < 0$, we have $\partial|\beta| = \{-1\}$ and thus set

  $$0 = \partial f(\beta) = 2(\beta - z) - \lambda,$$

  giving $\beta = z + \lambda/2$ (this is valid if $z < -\lambda/2$).

- At $\beta = 0$, we have $\partial|\beta| = [-1, 1]$. Thus $0 \in \partial f(\beta)$ if and only if

  $$0 \in -2z + \lambda[-1, 1] \iff |z| \leq \frac{\lambda}{2}.$$

Thus we have

$$\hat{\beta} = \operatorname{sgn}(z) \max\left(|z| - \frac{\lambda}{2}, 0\right).$$

Example 4.10 (Finding the LASSO estimator using gradient descent).

$$f(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|_1.$$

- Initialize $\beta^0 = 0$.
- Until convergence:

- $g := X^\intercal (y - X\beta)$.
- $z = \beta - \eta g$.
- $\beta_j^{\text{new}} := \operatorname{sgn}(z_j) \cdot \max\left(|z_j| - \eta\lambda, 0\right)$ for $j = 1, \ldots, p$.
- If $\|\beta^{\text{new}} - \beta\| \leq \varepsilon$, return β^{new}.
- Else, set $\beta := \beta^{\text{new}}$ and repeat.

4.3 Stochastic Gradient Descent

In general ML we have a **loss function** $L(\theta)$ and we take gradients with respect to θ, usually of the form

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \ell(\theta; x_i, y_i).$$

Example 4.11. $\theta = \beta$, $\ell(\theta; x_i, y_i) = (y_i - x_i\beta)^2$, etc.

To run gradient descent, we set

$$\begin{aligned} \theta &= \theta - \eta \nabla L(\theta) \\ &= \theta - \eta \frac{1}{n} \sum_{n} \nabla_\theta \ell(\theta; x_i, y_i). \end{aligned}$$

Note that we update the gradient once and sum over all (x_i, y_i).

Stochastic Gradient Descent (SGD): Instead, choose one random data point (x_i, y_i) and update the gradient many times.

$$\theta := \theta - \eta \nabla_\theta \ell(\theta; x_i, y_i).$$

Note that on average, we will converge to the same minimum:

$$\begin{aligned} \mathbb{E}\left[\nabla_\theta \ell(\theta; x_i, y_i)\right] &= \sum \mathbb{P}[\text{choose sample } i] \nabla_\theta \ell(\theta; x_i, y_i) \\ &= \frac{1}{n} \sum_{n} \nabla_\theta \ell(\theta; x_i, y_i) = \nabla L(\theta). \end{aligned}$$

Pseudocode for SGD:

- Initialize $\theta = \theta_0$. Let T be the number of **epochs** (1 epoch = i pass through all n data points), indexed by $t = 1, \ldots, T$. For each t, let η_t be the learning rate.
- For $t = 1, \ldots, T$:
  - Shuffle data.
  - For (x_i, y_i) in data (equivalently, we select i without replacement from $\{1, \ldots, n\}$):,
    - $g := \nabla_\theta \ell(\theta; x_i, y_i)$.

    * $\theta = \theta - \eta_t g$.

- Return θ.

Remark 4.12. Ordinary GD:

- 1 epoch = 1 update for θ.
- Deterministic.
- Good for datasets of fixed size.

SGD:

- 1 epoch = n updates for θ.
- Averages out to convergent values across iterations.
- Good for online algorithms (more data comes in).
- We approximate the gradient using Monte Carlo sampling.

4.4 Mini-batch Stochastic Gradient Descent

We can also sample function subsets of datasets (mini-batches). **Mini-batch SGD**: For a randomly chosen batch $B \subseteq \{1, \ldots, n\}$, we set

$$g_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla_\theta \ell(\theta; x_i, y_i).$$

Then update θ as

$$\theta := \theta - \eta \cdot g_B(\theta).$$

We proceed as follows:

- At start of epoch, randomly shuffle data set and split into consecutive chunks of size $|B|$, each chunk being a mini-batch.
- Each data point gives random direction but averages out over batches.

4.5 Online Algorithms

There are algorithms that depend on data of *fixed* size. **Online algorithms** are meant for **data steams** (new data keeps coming in).

Example 4.13 (Offline vs Online Median). Fix an array of size n. Find the medium.
Suppose now that data keeps coming in. How to find median at any given time?

Example 4.14. Given fixed set of data (x_i, y_i), $i = 1, \ldots, n$. Do linear regression to find

$$\hat{\beta} = \frac{1}{n} \sum \frac{(x_i - \overline{x})(y_i - \overline{y})}{(x_i - \overline{x})^2}.$$

If new data comes in, how to efficiently update $\hat{\beta}$? For a hint, recall $\hat{\beta}$ is the sample analogue of

$$\frac{\mathbb{C}(X, Y)}{\mathbb{V}(X)} = \frac{\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]}{\mathbb{E}[X^2] - (\mathbb{E}X)^2}.$$

4.6 Runtime

Let f, g be functions, with $g \geq 0$. We say $f(x) = O(g(x))$ if there exists $M > 0$ such that

$$|f(x)| \leq Mg(x).$$

Example 4.15. What is the runtime (= number of operations) of matrix multiplying a vector? $X \in \mathbb{R}^{n \times p}$ and $\beta \in \mathbb{R}^{p \times 1}$. For each row of X, we need p multiplications and $p - 1$ additions. Thus total number of operations is $n(2p - 1) = O(np)$.

Example 4.16.

- $O(p) + O(p) = O(p)$.
- $O(n) + O(n \log n) = O(n \log n)$.

Exercise 4.17 (2-sum). Given target value and an array of integers, find values x, y in the array such that $x + y =$ target.

Exercise 4.18 (Majority element). Given an integer array with a majority element (occurs more than $n/2$ times). How to find the majority element? Using a dictionary gives $O(n)$ space and time. It turns out there is also a $O(1)$ space and $O(n)$ time solution.

5 Singular Value Decomposition

A **singular value decomposition** (SVD) of A is a factorization $A = U\Sigma V^\top$ where

- $U \in \mathbb{R}^{m\times m}$ orthogonal ($U^\top U = \mathrm{Id}$) ($U \in \mathbb{C}^{m\times m}$ unitary, $U^* U = \mathrm{Id}$).
- $V \in \mathbb{R}^{n\times n}$ orthogonal ($in\mathbb{C}^{n\times n}$ unitary).
- $\Sigma \in \mathbb{R}^{m\times n}$ diagonal. Diagonal entries of Σ are nonnegative and in nonincreasing order:

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & 0 & \cdots & 0 \\ 0 & 0 & \sigma_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma_p \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix},$$

  where $p = \min(m, n)$ and $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$. The σ_i are called the **singular values** of A.

See Lecture 4–5 of textbook.

Remark 5.1.

$$A = \underbrace{U}_{\text{rotates/reflects}} \overbrace{\Sigma}^{\text{stretches \& scales sphere to ellipsoid}} \underbrace{V^\top}_{\text{rotates unit sphere}}.$$

Thus $V^\top$ maps the unit sphere to another sphere, Σ stretches and scales it to an ellipsoid, and U rotates/refects the ellipsoid. ☕

Theorem 5.2. *Every matrix $A \in \mathbb{R}^{m\times n}$ has an SVD with $\{\sigma_i\}$ uniquely determined. If A is square and σ_j are distinct, then $\{u_j\}$ and $\{v_j\}$ are also uniquely determined up to sign.*

Recall the **eigendecomposition** of a matrix $A \in \mathbb{R}^{n\times n}$ with n linearly independent eigenvectors is

$$A = Q\Lambda Q^{-1},$$

where Λ is diagonal with $\Lambda_{jj} = \lambda_j$ and columns of Q are eigenvectors of A.

Theorem 5.3 (Spectral Theorem)**.** *If A is symmetric ($A = A^\top$), then eigendecomposition exists and eigenvectors can be chosen to be orthonormal, i.e., $A = Q\Lambda Q^\top$.*

Theorem 5.4. *Nonzero singular values of A are square roots of nonzero eigenvalues of $A^\top A$ (or $AA^\top$).*

Proof.

$$\begin{aligned} A^\top A &= (U\Sigma V^\top)^\top (U\Sigma V^\top) \\ &= V\Sigma^\top \underbrace{U^\top U}_{\text{Id}} \Sigma V^\top \\ &= V \underbrace{\Sigma^\top \Sigma}_{n \times n \text{ diagonal}} V^\top = V\Sigma^2 V^\top . \end{aligned}$$

Note that Σ^2 is a diagonal matrix with entries squares of singular values of A. □

Theorem 5.5. *If A is symmetric, then singular values of A are absolute values of the eigenvalues of A.*

Proof. Recall that the singular values of A are square roots of eigenvalues of $A^\top A$. Let $A = Q\Lambda A^\top$ be the eigendecomposition of A. Then

$$A^\top A = Q\Lambda^\top Q^\top Q\Lambda Q^\top = Q\Lambda^2 Q^\top .$$

Thus the singular values of A are $\sqrt{\lambda_i^2} = |\lambda_j|$. □

6 Principal Component Analysis

Goal: given sample points in a high-dimensional $\mathbb{R}^d$, find k directions that capture most of the variance in the data. This is an example of **dimensionality reduction**.

Reasons?

- Reducing the number of dimension makes computation cheaper.
- Removes irrelevant dimensions to prevent overfitting.
- Gives smaller basis for representing variance.

Example 6.1 (Bad example). Given random data x_i, find a hyperplane (which can be identified with a normal vector $\mathbf{n}$) that best approximates it.

Goal: minimize

$$L(\mathbf{n}) = \sum_i \left(\mathbf{n}^\top (x_i - \overline{x})\right)^2 \quad \text{s.t.} \quad \|\mathbf{n}\| = 1.$$

We can rewrite the objective as

$$\begin{aligned} L(\mathbf{n}) &= \sum_i \mathbf{n}^\top (x_i - \overline{x})(x_i - \overline{x})^\top \mathbf{n} \\ &= \mathbf{n}^\top \left[\sum (x_i - \overline{x})(x_i - \overline{x})^\top\right] \mathbf{n} \\ &= \mathbf{n}^\top X^\top X \mathbf{n}, \end{aligned}$$

where X is the data matrix with rows $x_i - \overline{x}$. The Lagrangian is

$$\mathscr{L}(\mathbf{n}) = \mathbf{n}^\top X^\top X \mathbf{n} - \lambda(\mathbf{n}^\top \mathbf{n} - 1).$$

We have

$$\nabla_{\mathbf{n}} \mathscr{L} = 2X^\top X \mathbf{n} - 2\lambda \mathbf{n} = 0,$$

giving

$$X^\top X \mathbf{n} = \lambda \mathbf{n}.$$

That is, $\mathbf{n}$ is the eigenvector of $X^\top X$ with the smallest eigenvalue, which corresponds to the smallest singular value of X. Geometrically, the hyperplane normal to the minor axis of the ellipsoid formed by X best fits the data.

Remark 6.2. We could alternatively pick a direction to be y_i and use linear regression. The difference between linear regression and PCA is that linear regression minimizes vertical distances whereas PCA minimizes orthogonal distances to the hyperplane.

Let $X \in \mathbb{R}^{n \times p}$ be n data points with p features. We want to reduce the number of features but keep as much of variance as possible.

Goal: we want p vectors $w_j \in \mathbb{R}^k$ such that $(x_i^\top w_j) w_j$ (this is the orthogonal projection of x_i onto w_j) has maximum variance (norm), with $\|w_j\| = 1$. The norm here is

$$\left\langle (x_i \cdot w_j) w_j, (x_i \cdot w_j) w_j \right\rangle = (x_i \cdot w_j)^2 \|w_j\|^2 = (x_i \cdot w_j)^2.$$

To find the first principal component, we optimize

$$\max_w \sum_i (x_i^\top w)^2 \quad \text{s.t.} \quad \|w\| = 1.$$

The Lagrangian is

$$\mathscr{L}(w) = (Xw)^\top Xw - \lambda(w^\top w - 1).$$

We have

$$\nabla_w \mathscr{L} = 2X^\top Xw - 2\lambda w = 0 \implies X^\top Xw = \lambda w.$$

Thus $(Xw)^\top Xw$ is maximized over $\|w\| = 1$ when w is the eigenvector corresponding to the largest eigenvalue of $X^\top X$, the sample covariance matrix. Note that $w_{(1)}$ is the maximizer of the **Rayleigh quotient**:

$$w_{(1)} = \arg\max_w \frac{w^\top X^\top Xw}{w^\top w}.$$

To find the second principal component, let $X_{(2)} := X - Xw_{(1)}w_{(1)}^\top = X(\mathrm{Id} - w_{(1)}w_{(1)}^\top)$ (this is the projection to the orthogonal complement) and repeat on $X_{(2)}$.

Remark 6.3. Algorithm:

- Input: $X_1, \ldots, X_n \in \mathbb{R}^d$.
- Compute $S := X^\top X$.
- Take the eigendecomposition $S = PDP^\top$, where the column vectors of P are $v_1, \ldots, v_n$.
- Set $P_k := (v_1, \ldots, v_k)$.
- Output $Y_1, \ldots, Y_k$, where $Y_i := P_k^\top x_i \in \mathbb{R}^k$.

7 QR Factorization

Let A be a matrix with column vectors $a_1, \ldots, a_n$. Consider the increasing sequence of subspaces

$$\langle a_1 \rangle \subset \langle a_1, a_2 \rangle \subset \ldots$$

The idea of QR is to find orthonormal vectors $\{q_i\}$ that span these subspaces:

$$\left\langle q_1, \ldots, q_j \right\rangle = \left\langle a_1, \ldots, a_j \right\rangle$$

so that

$$A = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix} = \begin{pmatrix} q_1 & \cdots & q_n \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \end{pmatrix} = QR.$$

This allows us to go back and forth between the bases $\{a_i\}$ and $\{q_i\}$. For instance,

$$\begin{aligned} a_1 &= r_{11} q_1, \\ a_2 &= r_{12} q_1 + r_{22} q_2, \\ a_n &= r_{1n} q_1 + r_{2n} q_2 + \cdots + r_{nn} q_n. \end{aligned}$$

We can find q_i and r_{ij} using the **Gram-Schmidt algorithm**:

- At the jth step, we want $q_j \in \left\langle a_1, \ldots, a_j \right\rangle$ and is orthogonal to $q_1, \ldots, q_{j-1}$. Note that

  $$v_j \coloneqq a_j - (q_1^\intercal a_j) q_1 - (q_2^\intercal a_j) q_2 - \cdots - (q_{j-1}^\intercal a_j) q_{j-1}$$

  is orthogonal to $q_1, \ldots, q_{j-1}$. To see this, note for instance that

  $$q_1^\intercal v_j = q_1^\intercal a_j - (q_1^\intercal a_j)(q_1^\intercal q_1) - 0 - \cdots - 0 = 0.$$

  Note also that $r_{ij} = q_i^\intercal a_j$, so

  $$v_j = a_j - \sum_{i=1}^{j-1} (q_i^\intercal a_j) q_i \implies a_j = v_j + \sum_{i=1}^{j-1} (q_i^\intercal a_j) q_i.$$

  This implies $v_j = r_{jj} q_j$, or r_{jj} is the norm of v_j.

  We normalize v_j to get q_j:

  $$q_j = \frac{v_j}{\|v_j\|} = \frac{a_j - \sum_{i=1}^{j-1} (q_i^\intercal a_j) q_i}{\|a_j - \sum_{i=1}^{j-1} (q_i^\intercal a_j) q_i\|} = \frac{a_j - \sum_{i=1}^{j-1} (q_i^\intercal a_j) q_i}{r_{jj}}.$$

- Thus Gram-Schmidt gives:

$$
\begin{aligned}
q_1 &= \frac{a_1}{r_{11}}, \\
q_2 &= \frac{a_2 - (q_1^\intercal a_2) q_1}{r_{22}}, \\
&\vdots \\
q_n &= \frac{a_n - \sum_{i=1}^{n-1} (q_i^\intercal a_n) q_i}{r_{nn}}.
\end{aligned}
$$

8 Interpolation

Problem. Given $f \in C[a,b]$ and $n+1$ distinct points $x_0, \dots, x_n \in [a,b]$, find a polynomial p_n of degree at most n such that

$$p_n(x_i) = f(x_i), \quad i = 0, \dots, n.$$

Remark 8.1. This is useful for say integration: for a complicated f, we can approximate it with a polynomial p_n and integrate p_n instead. ☕

Proposition 8.2. *Existence, uniqueness.*

Proof. For existence, use a dimension argument. For uniqueness, note that if p_n, q_n solves interpolation, then $p_n - q_n$ has $n+1$ roots, so is identically 0. □

One way to solve this problem is to solve the linear system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix},$$

where $p_n(x) = a_0 + a_1 x + \cdots + a_n x^n$. The matrix here is a **Vandermonde matrix**.

This is using the most naive basis.

8.1 Newton Basis

The **newton basis** is

$$\begin{aligned} N_0(x) &= 1, \\ N_1(x) &= (x - x_0), \\ N_2(x) &= (x - x_0)(x - x_1), \\ &\vdots \\ N_n(x) &= (x - x_0)(x - x_1) \cdots (x - x_{n-1}). \end{aligned}$$

Each time, we attach a new polynomial factor that is 0 at the previous interpolation points.

Thus in general, we have

$$p_n(x) = p_{n-1}(x) + c_n \prod_{j=0}^{n-1} (x - x_j),$$

where c_n can be evaluated by setting $p_n(x_n) = f(x_n)$. This is essentially forcing our A matrix to be triangular, so we can solve it using back substitution instead of inverting

the matrix with time $O(n^3)$. Explicitly, we are solving

$$\begin{pmatrix} N_0 & 0 & 0 & \cdots & 0 \\ N_0 & N_1 & 0 & \cdots & 0 \\ N_0 & N_1 & N_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ N_0 & N_1 & N_2 & \cdots & N_n \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

8.2 Lagrange Basis

We have improved solving the system of equations from a dense matrix A to an upper triangular matrix. What if we wanted a diagonal A?

Consider a basis of the form

$$\ell_j(x_i) = \prod_{j \neq i} \frac{(x - x_j)}{(x_i - x_j)} = \begin{cases} 0, & i \neq j, \\ 1, & i = j. \end{cases}.$$

It is then immediate that

$$p_n(x) = \sum_{j=0}^{n} f(x_j)\ell_j(x).$$

8.3 Interpolation Error Bounds

Define the error as

$$E_n(x) \coloneqq f(x) - p_n(x).$$

Since $E_n(x_i) = 0$, let's write

$$E_n(x) = r(x) \prod_{i=0}^{n} (x - x_i)$$

for some function r. Choose $\hat{x} \in [a, b]$ such that $\hat{x} \neq x_i$ for each i and define

$$W(x) \coloneqq f(x) - p_n(x) - r(\hat{x}) \prod_{i=0}^{n} (x - x_i).$$

Note that $W(x_i) = 0$ for $0 \leq i \leq n$ and $W(\hat{x}) = 0$. In particular, W has at least $n + 2$ distinct roots. Using the mean value theorem and induction, we know that there is some $\xi \in [a, b]$ such that $W^{(n+1)}(\xi) = 0$.

Since

$$W^{(n+1)}(x) = f^{(n+1)}(x) - r(\hat{x}) \cdot (n + 1)!,$$

we have

$$r(\hat{x}) = \frac{f^{(n+1)}(\xi)}{(n + 1)!}.$$

Thus we have the following:

Theorem 8.3 (Interpolation Error Formula, Cauchy)**.** *Suppose $f \in C^{n+1}[a,b]$ and let p_n denote the f-interpolating polynomial with distinct nodes $x_0, \ldots, x_n \in [a,b]$. Then for every $x \in [a,b]$ there exists $\xi \in [a,b]$ such that*

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

Thus, the worst error over $[a,b]$ can be bounded as

$$\max_{x \in [a,b]} |f(x) - p_n(x)| \leq \left(\max_{\xi \in [a,b]} \frac{|f^{(n+1)}(\xi)|}{(n+1)!} \right) \left(\max_{x \in [a,b]} \prod_{j=0}^{n} |x - x_j| \right).$$

Remark 8.4.

- We have a better fit for "flatter" f.
- We have better fit near center since that makes $(x - x_0) \ldots (x - x_n)$ smaller.
- Error is unbounded on a unbounded set, since $(x - x_0) \ldots (x - x_n)$ can be arbitrarily large. We will naturally want to minimize this product. This problem will be revisited.

Remark 8.5. As x_n approaches 0, p_n approaches the Taylor polynomial of degree n at 0:

It turns out we have the following:

Theorem 8.6 (Chebyshev Interpolation)**.** *The choice of real numbers $x_0, \ldots, x_n \in [-1,1]$ that minimizes $\max_{x \in [-1,1]} |W(x)|$ is given by*

$$x_j = \cos\left(\frac{(2j+1)\pi}{2n} \right), \quad j = 0, \ldots, n.$$

Furthermore, the minimum value is $2^{-(n-1)}$.

9 Spline Interpolation

The idea is to use piecewise polynomials (**splines**) to interpolate data. Note that the resulting function is not necessarily differentiable at the knots.

Definition 9.1 (The Linear case)**.** Given $n+1$ data points $\{(x_j, f_j)\}$, we want n linear polynomials $\{s_j\}$ such that

$$s_j(x_{j-1}) = f_{j-1}, \quad s_j(x_j) = f_j, \quad j = 1, \ldots, n.$$

Note that we have

$$s_j(x) = f_j - \frac{f_j - f_{j-1}}{x_j - x_{j-1}}(x_j - x),$$

each s_j being defined on $[x_{j-1}, x_j]$.

Theorem 9.2. *Is σ is a function that interpolates $f(x)$ at $n+1$ distinct points $x_0 < \cdots < x_n$ and if $\|\sigma'\|_2$ is finite on $[x_0, x_n]$, then*

$$\|s'\|_2 \leq \|\sigma'\|_2,$$

where s is the linear spline interpolant of f at $\{x_j\}$. Thus the linear spline is the "flattest" among all interpolants.

Proof. Note from $[\sigma']^2 = [(\sigma - s)' + s']^2$, we have

$$\|\sigma'\|^2 = \|\sigma' - s'\|^2 + \|s'\|^2 + 2\int (\sigma - s)' s' \, \mathrm{d}x.$$

We rewrite the last term using integration by parts on each interval $[x_{i-1}, x_i]$ as

$$\int_{x_{i-1}}^{x_i} (\sigma - s)' s' \, \mathrm{d}x = \left[s' \int (\sigma - s)' \, \mathrm{d}x \right]_{x_{i-1}}^{x_i} - \int s'' \left(\int (\sigma - s)' \right) \mathrm{d}x = 0.$$

The last equality follows since σ and s agree at the knots and $s'' = 0$ on each interval. Thus $\|\sigma'\| \geq \|s'\|^2$. □

9.1 Cubic Splines

We can also use cubic splines between every two knots, with the additional requirement that the first and second derivatives agree at one end of each interval. With $n+1$ data points, we have $4(n-1)+1+1 = 4n-2$ requirements (4 per interval, 1 for left endpoint, 1 for right endpoint). This is no enough to specify a unique cubic spline.

10 Approximation Theory

Definition 10.1. Let (X, d) be a metrics space and $U \subseteq X$ be a subspace. A best approximation to $f \in X$ from U is an element $u^* \in U$ such that

$$d(f, u^*) = \inf\{d(f, u), u \in U\}.$$

Example 10.2. Let $X = C^0(I)$ and $U = P_n$. Then p^* exists. Consider a minimizing sequence p_n that (WLOG) includes 0. Note that the best approximation must lie in the set

$$\{p \in P_n : d(f, p) \leq d(f, 0)\},$$

which is a closed and bounded subset of $P_n \cong \mathbb{R}^{n+1}$. Note that

$$d(f, p^*) \leq d(f, p_i) + d(p_i, p^*).$$

As $p_i \to p^*$, we have

$$d(f, p^*) \leq \inf\{d(f, p_i)\} \leq d(f, p^*).$$

Definition 10.3 (Inner Product)**.** For functions $f, g \in C[a, b]$, we define the **inner product** of f and g as

$$\langle f, g\rangle = \int_a^b f(x)g(x)w(x)\,\mathrm{d}x,$$

where the weight function w is positive and continuous on (a, b), and integrable on $[a, b]$. We say f and g are **orthogonal** if $\langle f, g\rangle = 0$.

Theorem 10.4 (Gram-Schmidt)**.** *Given a finite-dimensional inner product space with a set of linearly independent vectors $\{\beta_1, \ldots, \beta_n\}$, we can always construct a set of orthogonal vectors $\{\alpha_1, \ldots, \alpha_n\}$ such that $\{\alpha_1, \ldots, \alpha_i\}$ is a basis for* $\operatorname{span}\{\beta_1, \ldots, \beta_i\}$ *for $i = 1, \ldots, n$.*

We have $\alpha_1 = \beta_1$. The orthogonal projection of β_2 on β_1 is given by

$$\frac{\langle \beta_2, \beta_1\rangle}{\|\beta_1\|^2}\beta_1.$$

Thus one can take $\alpha_2 = \beta_2 - \frac{\langle \beta_2, \beta_1\rangle}{\|\beta_1\|^2}\beta_1$. The rest is similar, with

$$\alpha_3 = \beta_3 - \text{projection of } \beta_3 \text{ on } \operatorname{span}\{\alpha_1, \alpha_2\},$$

and so on.

Using the Gram-Schmidt algorithm, we can construct an orthogonal basis for P_n with respect to any inner product, using any basis of P_n (e.g., the monomial basis).

Proposition 10.5. *Given $f \in C[a,b]$, the polynomial $p^* \in \mathcal{P}_n$ minimizing $\|p - f\|_2$ among all $p \in \mathcal{P}_n$ if and only if*

$$\langle p^* - f, p \rangle = 0, \quad \forall p \in \mathcal{P}_n.$$

In other words, p^* is a best approximation to f if and only if $p^* - f$ is orthogonal to $\mathcal{P}_n$.

Proof. We have

$$\|f - p\|^2 \leq \|f - p^*\|^2 + \|p^* - p\|^2,$$

with equality if and only if $\langle p^* - f, p^* - p \rangle = 0$. □

With this in mind, we have if $p^* = \sum c_i \varphi_i$, then it must be the case that

$$\langle p^* - f, \varphi_i \rangle = 0 \implies c_i = \langle f, \varphi_i \rangle .$$

Proposition 10.6. *The best approximation to f from P_n with respect to the inner product $\langle \cdot, \cdot \rangle$ is given by*

$$Lf := \sum_{i=0}^{n} \langle f, \varphi_i \rangle \varphi_i,$$

where $\{\varphi_i\}$ is an orthonormal family of polynomials with respect to the inner product.

11 Numerical Quadrature

Quadrature is just a fancy name for numerical integration.

All Riemann sums can be written as $\sum w_i f(x_i)$. How do we choose w_i and x_i? We use $I(f)$ to denote the true value of the integral and $Q(f) = \sum w_i f(x_i)$ to denote the quadrature approximation.

Proposition 11.1. *If $f \in C^1[a,b]$, then*

$$|I(f) - Q_L(f)| \leq \frac{b-a}{2}\|f'\|_\infty h,$$

where Q_L denotes the left Riemann sum.

Proof.

$$\begin{aligned}
|I - Q| &= \sum \int_{x_i}^{x_{i+1}} |f(x) - f(x_i)| \, \mathrm{d}x \\
&= \sum \int_{x_i}^{x_{i+1}} (x - x_i)|f'(\xi_i)| \, \mathrm{d}x \\
&\leq \sum \int_{x_i}^{x_{i+1}} \|f'\|_\infty (x - x_i) \, \mathrm{d}x \\
&= \|f'\|_\infty \sum \frac{(x_{i+1} - x_i)^2}{2} = \|f'\|_\infty \frac{h}{2} \sum (x_{i+1} - x_i) = \|f'\|_\infty \frac{h}{2}(b-a).
\end{aligned}$$

□

Trapezoidal rule:

$$Q_T(f) := \sum \frac{h_i}{2}(f(x_i) + f(x_{i+1})) = \frac{h}{2}\left(f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\right).$$

This can equivalently be viewed as integrating the linear spline interpolant of f at the knots. The left and right Riemann sums can be viewed as integrating the piecewise constant spline interpolant of f at the knots.

Note also that on a uniform mesh, the trapezoidal rule is the average of the left and right Riemann sums.

Proposition 11.2. *If $f \in C^2[a,b]$, then*

$$|I(f) - Q_T(f)| \leq \frac{b-a}{12}\|f'\|_\infty h^2.$$

Thus the error converges much faster, with order h^2.

We can of course use higher order splines. The **Newton-Cotes quadrature** is baed on choosing Lagrange interpolation with equally spaced notes.

11.1 Simpson's Rule

Simpson's rule is based on using quadratic splines.

11.2 Gaussian Quadrature

Fix weighting function $w(x)$. Let $\{\varphi_i\}$ be the orthogonal family of polynomials with respect to the inner product $\langle \cdot, \cdot \rangle$ defined by w. Then the roots of φ_{n+1} are the optimal choice of nodes for quadrature with $n+1$ nodes, and the corresponding weights are given by

$$w_i := \int \ell_i(x) w(x) \, \mathrm{d}x$$

where ℓ_i is the Lagrange basis polynomial corresponding to the ith root of φ_{n+1}.

With Gauss-Legrendre quadrature ($w \equiv 1$), plotting the nodes against weights, they lie on a semicircle. With Gauss-Chebyshev quadrature ($w(x) = (1 - x^2)^{-1/2}$), the φ_n are scaled Chebyshev polynomials, and the weights are the same for all nodes, and the nodes are more dense near the endpoints.

11.3 Quadrature on General Intervals

To eluate an integral over a general interval $[a, b]$, we apply the affine transformation $x \mapsto \frac{b-a}{2} x + \frac{a+b}{2}$ to transform the nodes, and scale the weights by $\frac{b-a}{2}$.

11.4 Degree of Exactness

Definition 11.3. The **degree of exactness** of a quadrature rule Q is the largest integer d such that $Q(p) = I(p)$ for all polynomials p of degree at most d.

Why is this important? Let f be an arbitrary function and let p be a polynomial for which Q is exact. Then

$$Q(f) - I(f) = Q(f - p) - I(f - p).$$

Theorem 11.4. *Let w be nonnegative and integrable on $[a, b]$, and suppose the quadrature rule*

$$Q(f) := \sum_{j=0}^{n} w_j f(x_j)$$

is exact for al polynomials in $\mathcal{P}_k$. Then for every $f \in C[a, b]$ and every $p \in \mathcal{P}_k$,

$$|Q(f) - I(f)| \leq \left(\sum |w_j| + \int w \, \mathrm{d}x \right) \|f - p\|_\infty.$$

Proof.

$$\begin{aligned} |Q(f) - I(f)| &= \left| \sum w_j [f(x_j) - p(x_i)] - \int (f - p) w \, \mathrm{d}x \right| \\ &\leq \sum |w_j| \|f - p\|_\infty + \int |f - p| w \, \mathrm{d}x \\ &\leq \left(\sum |w_j| + \int w \, \mathrm{d}x \right) \|f - p\|_\infty. \end{aligned}$$

□

The reason why quadratures might be noisy for high degrees is that $\sum |w_j|$ may not converge. In the case of Gaussian quadrature, the weights are all positive and sum to $\int w \, \mathrm{d}x$, so the error is bounded by $2 \int w \, \mathrm{d}x \cdot \|f - p\|_\infty$.

Theorem 11.5. *The Gaussian quadrature with roots $\{x_i\}$ of φ_{n+1} is exact for all polynomials of degree at most $2n + 1$.*

Proof. Given $p \in \mathcal{P}_{2n+1}$, find $q, rin\mathcal{P}_n$ such that $p = q\varphi_{n+1} + r$. We have

$$\int_a^b pw \, \mathrm{d}x = \int rw \, \mathrm{d}x = \sum_{i=0}^{n} w_i r(x_i) = \sum_i w_i p(x_i).$$

□

Now, because in particular the quadrature is exact for ℓ_k, we have

$$w_k = \int \ell_k^2 w \, \mathrm{d}x.$$

This formula is more computationally reliable and shows that w_k are positive.

Let's return to the analysis of Gaussian quadrature error. Let $p \in \mathcal{P}_{2n+1}$ to have the same value and the same first derivatives as f at the nodes. Write

$$E(x) \coloneqq f(x) - p(x).$$

Since E and E' are zero at each node, each node has multiplicity 2 as a root. Thus we can write

$$E(x) = r(x) \cdot \prod_{i=0}^{n} (x - x_i)^2.$$

Using a similar trick as before with a twisted $\hat{E}(x)$, we get

$$f(x) - p(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \prod_{i=0}^{n} (x - x_i)^2.$$

Note that $\prod (x - x_i)$ is the scaled φ_{n+1}, we have blah blah blah

12 Ordinary Differential Equations

The initial value problem is to solve the ODE

$$y' = f(t, y), \quad y(t_0) = y_0.$$

We will be interested in approximating $y(T)$ for some $T > t_0$.

12.1 Euler's Method and Heun's Method

$$y_{i+1} \approx y_i + hf(t_i, y_i) + \frac{h}{2}\left[f(t_{i+1}, y_i + hf(t_i, y_i), f(t_i, y_i)) - f(t_i, y_i)\right].$$

The idea behind this approximation is second order Taylor expansion of y at t_i,

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(t_i) + O(h^3)$$

and approximation of $y''(t_i)$ by

$$y''(t_i) \approx \frac{y'(t_{i+1}) - y'(t_i)}{h} \approx \frac{f(t_{i+1}, y_i + hf(t_i, y_i)) - f(t_i, y_i)}{h}.$$

We can also solve t_{i+1} implicitly in the equation

$$y_{i+1} = y_i + hf(t_i, y_i) + \frac{h^2}{2}\left[f(t_{i+1}, y_{i+1}) - f(t_i, y_i)\right].$$

This can be viewed as a Taylor approximation based method, but it is also integrating a linear spline interpolant of f. This is called **Heun's method**.

Remark 12.1. The constant splint method is precisely the left Riemann sum:

$$\begin{aligned} y(T) = y_0 + \int_{t_0}^{T} f(t, y(t))\,\mathrm{d}t &\approx y_0 + h\sum \int_{t_i}^{t_{i+1}} f(t, y(t))\,\mathrm{d}t \\ &\approx y_0 + h\sum f(t_i, y(t_i)). \end{aligned}$$

This is called the **Euler's method**.

12.2 Runge-Kutta Methods

We consider the approximation

$$\int_{t_i}^{t_{i+1}} f(t, y(t))\,\mathrm{d}t \approx \sum b_j k_j,$$

where the b_j's are weights. Consider nodes $t_i + c_j h$ for $1 \le j \le n$ with $t_{i+1} = t_i + c_n h$. The ks are defined as

$$\begin{aligned} k_1 &= f(t_i, y_i), \\ k_2 &= f(t_i + c_2 h, y_i + ha_{21}k_1), \\ &\vdots \\ k_n &= f(t_i + c_n h, y_i + h(a_{n1}k_1 + \cdots + a_{n,n-1}k_{n-1})). \end{aligned}$$

12.3 Multistep Methods

13 MISC

Proposition 13.1. *A is diagonally dominant if and only if the iteration matrix for the Jacobi method* $T := D^{-1}(D - A) = I - D^{-1}A$ *satisfies* $\|T\|_\infty < 1$.

Proof. Recall $\|T\|_\infty = \max_i \sum_j |t_{ij}|$, and

$$\left| \sum_j \frac{a_{ij}}{a_{ii}} \right| < 1$$

is equivalent to the condition for diagonal dominance. □

Proposition 13.2. $A^k \to 0$ *if and only if* $\rho(A) < 1$.

Proof. $\Leftarrow$: Find $\delta > 0$ such that $\rho(A) + \delta < 1$ and find norm such that $\|A\| \leq \rho(A) + \delta < 0$.

$\Rightarrow$: If $\rho(A) \geq 1$ so that there is an eigenvalue λ of A with $|\lambda| \geq 1$. Let u be the eigenvector corresponding to λ. Then $A^k u = \lambda^k u$ does not converge to 0. □

Index

class, 9
Classification, 9
Consistent, 6
convex, 17

data steams, 20
degree of exactness, 35
dimensionality reduction, 24

eigendecomposition, 22
epochs, 19
estimator, 6
Euler's method, 37

Gram-Schmidt algorithm, 26

Heun's method, 37

inner product, 32

jointly Gaussian, 7

least squares error, 10
likelihood, 5
log-likelihood, 5
loss function, 19

maximum a posteriori, 16
maximum likelihood estimation, 5
mean squared error, 6
Mini-batch SGD, 20
MLE, 5
multivariate normal distribution, 7

newton basis, 28
Newton-Cotes quadrature, 34

Online algorithms, 20
orthogonal, 32
orthogonal projection, 13
overfitting, 14

parametric statistics, 4
projection matrix, 13

Quadrature, 34

Rayleigh quotient, 25
Regression, 9

singular value decomposition, 22
singular values, 22
splines, 31
Stochastic Gradient Descent, 19
subderivative, 18
subdifferential, 18

Unbiased, 6
unbiased, 6
underfitting, 14

Vandermonde matrix, 28